

PLAN DE ACCIÓN Y ROADMAP

Simulador de Diagnóstico — Versión Low-Cost

*Ruta concreta, con herramientas gratuitas o de bajo costo, para llegar a un producto accesible
para los estudiantes*

Preparado para: Academias Ethermic

Julio 2026

Índice

1. Resumen ejecutivo y filosofía low-cost
2. Principios de decisión
3. Stack definitivo (herramientas y costos)
4. Arquitectura simplificada del sistema
5. Roadmap por fases (acciones, tiempos, costo)
6. Plan de acceso y distribución para estudiantes
7. Presupuesto estimado total
8. Próximos pasos inmediatos (esta semana)

1. Resumen ejecutivo y filosofía low-cost

El objetivo de este plan es llegar a un simulador de diagnóstico propio y funcional gastando lo mínimo posible en herramientas, usando en su gran mayoría software gratuito o con planes gratuitos suficientes para esta etapa, y asegurando que el resultado final sea fácil de abrir para cualquier estudiante: un enlace web, sin instalar nada, que funcione en computadoras modestas y navegadores comunes.

La estrategia central es: construir primero en 2D web (barato, rápido, liviano) para validar que el motor de fallas y el sistema de puntuación funcionan bien en clase; solo si más adelante se justifica, invertir en una versión 3D más costosa. Esto evita gastar presupuesto grande en arte 3D antes de comprobar que la pedagogía funciona.

2. Principios de decisión

- Cero instalación para el estudiante: todo corre en el navegador (Chrome, Edge, Firefox), como página web normal.
- Gratis primero, de pago solo si es indispensable: se usan planes "free tier" de cada herramienta hasta que el número de usuarios lo justifique.
- Reutilizar lo que el equipo ya sabe usar: Supabase/PostgreSQL y Vercel, que ya se emplean en ZR Help, para no sumar curva de aprendizaje nueva.
- Liviano en recursos: pensado para que corra bien incluso en equipos de laboratorio antiguos, no solo en computadoras de última generación.
- Modular: separar el motor de simulación (la lógica) del arte visual, de modo que mejorar el aspecto gráfico en el futuro no obligue a rehacer el sistema.

3. Stack definitivo (herramientas y costos)

3.1 Desarrollo y diseño

Necesidad	Herramienta recomendada	Costo
Interfaz interactiva (drag & drop, zoom, capas)	React + Konva.js (o PIXIJS)	Gratis / open source
Diseño de pantallas e ilustraciones de componentes	Figma (plan gratuito) + Inkscape para vectores	Gratis
Editor de código	VS Code	Gratis
Control de versiones	GitHub (repositorio privado, plan gratuito)	Gratis
Gestión de tareas del equipo	Notion o Trello (plan gratuito)	Gratis

3.2 Backend, datos y autenticación

Necesidad	Herramienta recomendada	Costo
Base de datos (catálogo de fallas, componentes, intentos, puntajes)	Supabase (PostgreSQL) — mismo stack de ZR Help	Gratis hasta ~50.000 usuarios activos/mes en el plan free
Login de estudiantes/instructores	Supabase Auth	Incluido en el plan gratuito
Hosting de la aplicación web	Vercel (igual que ZR Help)	Gratis en el plan Hobby
Almacenamiento de imágenes/sprites	Supabase Storage	Incluido en el plan gratuito (límite de espacio generoso para 2D)

3.3 Assets visuales (2D)

- Ilustraciones propias en Figma/Inkscape, o adaptación de sprites/iconos libres de librerías como Flaticon, unDraw o Freepik (verificando licencia de uso comercial/educativo, muchas son gratis con atribución).
- Fotografías reales de componentes automotrices (tomadas por el equipo técnico) recortadas y vectorizadas: opción de costo cero y con mayor valor didáctico al ser piezas reales.

3.4 Solo si se decide escalar a 3D más adelante (fase opcional, no inicial)

- Modelado: Blender (gratis).
- Motor 3D web: Three.js o Babylon.js (gratis, open source).
- Modelos base para acelerar: bancos con planes gratuitos/económicos como Sketchfab (filtrar por licencia "Creative Commons" o "editorial/uso libre"), verificando siempre el permiso de uso educativo antes de integrarlos.

4. Arquitectura simplificada del sistema

Para minimizar costo y complejidad, el sistema se divide en tres piezas simples que se pueden construir una por una:

A. Motor de circuito (lógica)

Un archivo de datos (tabla en Supabase) que describe cada componente del sistema simulado: nombre, tipo (sensor/actuador/cable/módulo), rango normal de valores, y a qué otros componentes está conectado. Una función en el código (JavaScript/TypeScript) recorre esa tabla para calcular qué debería medir cada herramienta según el estado actual (normal o con una falla aplicada).

B. Catálogo de fallas (datos, editable sin programar)

Una tabla simple en Supabase donde cada fila es una falla: qué componente se altera, qué texto de "orden de trabajo" se le muestra al estudiante, el precio de la pieza y el nivel de dificultad. Esto permite that un instructor cargue fallas nuevas directamente en una tabla o un mini-formulario, sin tocar código — el mismo concepto que usa Electude con sus ~60 fallas, pero abierto a que ustedes agreguen las que quieran.

C. Interfaz (2D)

Las pantallas de: taller/vehículo con componentes clicables, panel de herramientas (multímetro, osciloscopio simplificado, escáner, orden de trabajo, factura), y panel de resultado final con el puntaje. Se construye con componentes React reutilizables, así agregar un nuevo "caso" no implica programar una pantalla nueva.

5. Roadmap por fases (acciones, tiempos, costo)

Fase	Acciones clave	Tiempo estimado	Costo herramientas
0. Preparación	Crear repositorio en GitHub, proyecto en Supabase, proyecto en Vercel, tablero de tareas en Notion/Trello, definir 1er sistema a simular (ej.: sistema de encendido básico)	3-5 días	\$0
1. Motor de circuito	Modelar en Supabase la tabla de componentes y conexiones; escribir la función que calcula lecturas según el estado del sistema	2-3 semanas	\$0
2. Catálogo de fallas	Tabla de fallas editable; 5-8 fallas iniciales bien documentadas para el primer sistema	1 semana	\$0
3. Prototipo jugable 2D	Pantalla del taller, multímetro básico, orden de trabajo, factura y puntaje funcionando de principio a fin para un solo caso	3-4 semanas	\$0
4. Osciloscopio y escáner virtual	Sumar las herramientas más avanzadas al prototipo ya probado	2-3 semanas	\$0
5. Cuentas, niveles y leaderboard	Login de estudiantes (Supabase Auth), guardado de intentos, tabla de posiciones por clase/academia	2 semanas	\$0
6. Editor de casos para instructores	Formulario simple para que un docente cargue una falla nueva sin programar	2 semanas	\$0
7. Piloto con un	Prueba real en clase, recolección de comentarios,	2-3 semanas	\$0

Fase	Acciones clave	Tiempo estimado	Costo herramientas
grupo de estudiantes	ajustes de dificultad y de puntuación		
8. (Opcional) Escalado a 3D	Solo si el piloto valida la pedagogía: modelar en Blender y migrar la interfaz a Three.js/Babylon.js	8-12 semanas	\$0 herramientas / posible costo si se contratan modelos 3D o un artista

Nota: los tiempos asumen 1-2 personas trabajando de forma parcial (no dedicación completa). Si el equipo puede dedicar tiempo completo, las fases 1 a 7 podrían comprimirse a 2-3 meses en total.

6. Plan de acceso y distribución para estudiantes

Pensado para que, una vez listo, cualquier estudiante entre sin fricción:

- Publicación en Vercel con un dominio propio simple (ej. simulador.ethermic.com), igual que ZR Help.
- La aplicación se construye como sitio web responsivo: funciona igual en computadora, tablet o celular, sin instalar nada.
- Opción de configurarla como PWA (Progressive Web App): el estudiante puede "agregar a inicio" en su celular y se siente como una app, sin pasar por tiendas de aplicaciones ni costos de publicación.
- Optimización de peso de imágenes y código (assets 2D comprimidos) para que cargue rápido incluso con internet limitado o en equipos de laboratorio más antiguos.
- Un solo enlace y usuario/contraseña (o acceso por código de clase) es todo lo que el estudiante necesita para empezar.

7. Presupuesto estimado total

Concepto	Costo mensual estimado
Herramientas de desarrollo (Figma, GitHub, VS Code, Notion)	\$0
Supabase (base de datos + autenticación + almacenamiento)	\$0 en fase inicial (plan gratuito); considerar plan Pro (~US\$25/mes) solo si el número de estudiantes crece mucho
Vercel (hosting)	\$0 en fase inicial (plan Hobby); plan Pro (~US\$20/mes) solo si se necesita uso comercial/equipo con más miembros
Dominio propio (opcional)	~US\$10-15 por año
Fase 3D opcional (si se decide más adelante)	Variable: \$0 si se modela internamente en Blender; con artista freelance, desde ~US\$300-800 según alcance

En resumen: se puede llegar a un prototipo jugable completo con costo de herramientas de \$0, dependiendo únicamente del tiempo del equipo. El único gasto casi seguro es el dominio propio (muy bajo), y los planes de pago de Supabase/Vercel solo aplican cuando el proyecto ya tenga tráfico real de muchos estudiantes — momento en el que, además, ya se habrá validado que vale la pena escalar.

8. Próximos pasos inmediatos (esta semana)

1. Elegir el primer sistema a simular (recomendado: algo acotado, ej. circuito de encendido o un sensor + actuador simple, no el motor completo).
2. Crear el repositorio en GitHub y el proyecto en Supabase (reutilizando la cuenta/organización ya usada en ZR Help si aplica).
3. Diseñar en una hoja simple (Notion o incluso papel) el diagrama de componentes y conexiones del sistema elegido, antes de tocar código.
4. Redactar 3 fallas de ejemplo con su descripción de "orden de trabajo", para tener casos de prueba desde el día uno.

5. Definir quién del equipo lidera cada capa: motor de circuito, interfaz 2D, contenido pedagógico (textos de fallas y checklist).

Cierre

Este plan prioriza avanzar rápido con herramientas gratuitas y sin fricción, dejando la puerta abierta a mejoras visuales (3D, IA de pistas, etc.) para una segunda etapa, una vez que el motor de fallas y el sistema de evaluación ya estén probados con estudiantes reales. Puedo ayudar a continuación con: el diseño exacto de las tablas en Supabase, el primer prototipo de código del motor de circuito, o la redacción de las primeras fallas y su checklist.